Sets and Dictionaries in Python

Contents

1 Introduction

Till now we learned:

- Variables
- Lists
- Tuples

But Python provides more powerful data structures.
Today we learn:

- Set
- Dictionary

These are extremely important in real-world programming.
Especially dictionaries.
Professional Python developers use dictionaries everywhere.
Sometimes so much that it starts feeling illegal.

2 Set

A set is:

Collection of unique unordered data.

Important properties:

- No duplicate values
- Unordered
- No indexing

3 Creating Set

Wrong way:

```
ram_hobbies = {}
```

This creates:

Dictionary

Correct way:

```
ram_hobbies = set()
```

Or:

```
ram_hobbies = {"cycling", "reading", "football"}
```

4 Duplicate Removal

```
ram_hobbies = {
    "cycling",
    "reading",
    "writing",
    "football",
    "reading"
}
```

Output:

```
{'cycling', 'reading', 'writing', 'football'}
```

Notice:

```
"reading"
```

appeared only once.
Sets automatically remove duplicates.
Python basically says:

"I already have this item. No need again."

5 Why Set Has No Indexing

Lists support indexing:

```
participants[0]
```

But sets do not.
Because sets are:

Unordered

Meaning:

- Items do not stay in fixed positions

So this is invalid:

```
ram_hobbies[0]
```

Python gives error.

6 Set Operations

Sets are heavily used in mathematics.
Python supports mathematical set operations directly.

7 Intersection

Intersection means:

Common items between sets.

Example:

```
ram_hobbies = {"cycling", "reading", "writing"}
hari_hobbies = {"cycling", "cricket", "writing"}
```

7.1 Method Form

```
commons = ram_hobbies.intersection(hari_hobbies)
```

7.2 Shortcut Form

```
commons = ram_hobbies & hari_hobbies
```

Output:

```
{'cycling', 'writing'}
```

8 Union

Union means:

All unique items from both sets.

8.1 Method Form

```
all_hobbies = ram_hobbies.union(hari_hobbies)
```

8.2 Shortcut Form

```
all_hobbies = ram_hobbies | hari_hobbies
```

Output:

```
{
 'cycling',
 'reading',
 'writing',
 'cricket'
}
```

9 Difference

Difference means:

Items present in first set but not in second set.

9.1 Example

```
ram_only = ram_hobbies - hari_hobbies
```

Output:

```
{'reading'}
```

Because:

- reading exists only in Ram's hobbies

9.2 Reverse Difference

```
hari_only = hari_hobbies - ram_hobbies
```

Output:

```
{'cricket'}
```

10 Symmetric Difference

Means:

Items that are NOT common.

10.1 Method Form

```
sym_diff = ram_hobbies.symmetric_difference(hari_hobbies)
```

10.2 Shortcut Form

```
sym_diff = ram_hobbies ^ hari_hobbies
```

Output:

```
{'reading', 'cricket'}
```

Common items removed.

11 Adding Data to Set

11.1 Adding One Item

```
ram_hobbies.add("rafting")
```

11.2 Adding Multiple Items

```
ram_hobbies.update(["swimming", "dancing"])
```

12 Removing Items from Set

12.1 remove()

```
ram_hobbies.remove("cycling")
```

If item does not exist:

- Python gives error

12.2 discard()

```
ram_hobbies.discard("swimming")
```

If item does not exist:

- No error

Much safer.

13 Dictionary

Dictionary stores:

Key-value pairs

Example:

```
person_info = {
    "name": "ram",
    "age": 23,
    "address": "Jhapa"
}
```

Think of dictionary like real-world dictionary.
Example:

```
word -> meaning
```

Similarly:

```
"name" -> "ram"
"age" -> 23
```

14 Accessing Dictionary Values

```
print(person_info["address"])
```

Output:

```
Jhapa
```

15 Dictionary with Multiple Data Types

```
person_info = {
    "name": "ram",
    "age": 23,
    "hobbies": {"cycling", "reading"},
    "address": "Jhapa"
}
```

Dictionary values can store:

- Strings
- Numbers
- Lists
- Sets
- Even another dictionary

Very powerful.

16 Dictionary Keys and Values

16.1 keys()

```
print(person_info.keys())
```

Output:

```
dict_keys(['name', 'age', 'address'])
```

16.2 values()

```
print(person_info.values())
```

17 Checking Key Existence

```
"name" in person_info.keys()
```

Output:

```
True
```

Useful for validation.

18 Problem with Direct Access

Suppose:

```
print(person_info["adddress"])
```

Notice spelling mistake:

```
adddress
```

Python gives:

```
KeyError
```

19 Safe Access Using get()

```
person_info.get("adddress", "No value found")
```

Output:

```
No value found
```

Much safer.

20 items() Method

```
print(person_info.items())
```

Output:

```
dict_items([
 ('name', 'ram'),
 ('age', 23)
])
```

21 Looping Through Dictionary

```
for key, value in person_info.items():
    print(key, value)
```

Output:

```
name ram
age 23
address Jhapa
```

22 Adding New Data

```
person_info["salary"] = 120000
```

Dictionary now contains new key-value pair.

23 Updating Existing Data

```
person_info["address"] = "KTM"
```

Old value replaced.

24 Removing Dictionary Data

24.1 pop()

```
person_info.pop("address")
```

Removes specific key.

24.2 popitem()

```
person_info.popitem()
```

Removes last inserted item.

24.3 del Keyword

```
del person_info["address"]
```

Deletes item completely.

25 Dictionary Keys Must Be Immutable

Keys can be:

- String
- Integer
- Tuple

Keys cannot be:

- List
- Set

Because mutable objects cannot become dictionary keys.

26 Nested Dictionary

Dictionaries can contain another dictionary.
Example:

```
person_info = {
    "name": "ram",
    "address": {
        "permanent": "ktm",
        "temporary": {
            "city": "biratnagar",
            "street": "st_10001"
        }
    }
}
```

This is called:

Nested Dictionary

27 Real API-like Data

Real applications often return huge nested data.
Example:

```
user_addresses = {
    "count": 2,
    "results": [
        {
            "city": "Tirakharda",
            "pincode": "854328"
        },
        {
            "city": "Gurugram",
```

```
            "pincode": "122001"
        }
    ]
}
```

28 Accessing Nested Data

```
print(user_addresses["results"][0]["pincode"])
```

Step-by-step:

- Get results list
- Get first dictionary
- Get pincode value

Output:

```
854328
```

29 Why Dictionaries are Extremely Important

Dictionaries are used everywhere:

- APIs
- JSON data
- Databases
- User profiles
- Authentication systems
- Configuration files

If lists are important, then dictionaries are absolutely essential.

30 Final Thoughts

Today you learned:

- Sets
- Unique collections
- Set operations
- Dictionaries

- Key-value pairs
- Nested dictionaries
- Dictionary methods

You are now learning data structures used in real software development. This is a major step forward.

31 Exercises

31.1 Exercise 1: Create Set

Create set of:

- 5 favorite foods

Print all items.

31.2 Exercise 2: Remove Duplicates

Take list with duplicate values.
Convert into set.

31.3 Exercise 3: Common Friends

Create two sets:

- Ram's friends
- Hari's friends

Find common friends.

31.4 Exercise 4: Unique Characters

Take sentence input.
Print unique characters using set.

31.5 Exercise 5: Student Dictionary

Create dictionary storing:

- Name
- Age
- Faculty

Print all data.

31.6 Exercise 6: Dictionary Loop

Loop through dictionary and print:

```
key -> value
```

31.7 Exercise 7: Update Data

Create employee dictionary.
Update salary.

31.8 Exercise 8: Safe Lookup

Ask user key name.

Use:

```
get()
```

to print value safely.

31.9 Exercise 9: Nested Dictionary

Create nested dictionary for:

- Student
- Address
- Contact

Print city.

31.10 Exercise 10: Word Counter

Take sentence input.

Count frequency of each word using dictionary.

32 Dictionary Practice Questions

32.1 Question 1: Student Result System

Create dictionary storing:

- Student name
- Marks in 3 subjects

Calculate:

- Total
- Percentage

32.2 Question 2: Phonebook System

Create phonebook dictionary:

```
name -> phone number
```

Menu options:

- Add contact
- Search contact
- Delete contact
- Show all contacts

32.3 Question 3: Voting System

Store votes inside dictionary.

Example:

```
{
    "Ram": 4,
    "Hari": 2
}
```

Print winner.

32.4 Question 4: Inventory System

Create dictionary:

```
product -> quantity
```

Update stock after purchase.

32.5 Question 5: Character Frequency

Take sentence input.

Count frequency of every character.

Example:

```
hello
```

Output:

```
h -> 1
e -> 1
l -> 2
o -> 1
```

32.6 Question 6: Employee Database

Store employees using nested dictionary.

Example:

```
{
    "emp1": {
        "name": "Ram",
        "salary": 50000
    }
}
```

32.7 Question 7: Quiz System

Store questions and answers in dictionary.

Ask questions to user.

Calculate score.

32.8 Question 8: Shopping Cart

Store products and prices in dictionary.

Calculate total bill.

32.9 Question 9: Country Capital Finder

Store:

```
country -> capital
```

Ask user country and print capital.

32.10 Question 10: Attendance System

Store attendance:

```
student -> present/absent
```

Print all absent students.

33 Challenge Questions

33.1 Challenge 1: Most Frequent Word

Take paragraph input.

Find most repeated word.

33.2 Challenge 2: Mini JSON Parser Feeling

Create deeply nested dictionary.

Practice accessing values using multiple indexes and keys.

33.3 Challenge 3: Leaderboard System

Store player scores.

Print top 3 players.

33.4 Challenge 4: Dictionary Merge

Merge two dictionaries manually without using update().

33.5 Challenge 5: Reverse Dictionary

Convert:

```
{
    "Ram": 98
}
```

into:

```
{
    98: "Ram"
}
```

33.6 Challenge 6: Duplicate Finder Using Dictionary

Take list of numbers.

Use dictionary to count duplicates.

33.7 Challenge 7: Login System

Store:

```
username -> password
```

Allow login validation.

33.8 Challenge 8: E-commerce Order System

Create nested dictionary for:

- User
- Orders
- Products
- Prices

Calculate total order amount.

33.9 Challenge 9: Student Ranking System

Store marks of students.

Print:

- Highest marks
- Lowest marks
- Average
- Rank list

33.10 Challenge 10: Real API Data Traversal

Create nested structure similar to:

```
{
    "results": [
        {
            "user": {
                "profile": {
                    "city": "Kathmandu"
                }
            }
        }
    ]
}
```

Practice accessing deeply nested values.

34 Homework Advice

Dictionaries become easy only after lots of practice.

At first nested dictionaries look terrifying.

Then one day you realize:

"Oh... it's just keys inside keys."

That is the moment real Python confidence starts growing.